



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE



Persistence Wears Down Resistance

Persistent Fault Analysis and Beyond on Fault Countermeasures

Shivam Bhasin
Temasek Labs @ NTU

SPACE 2018, Kanpur, India
19 Dec 2018



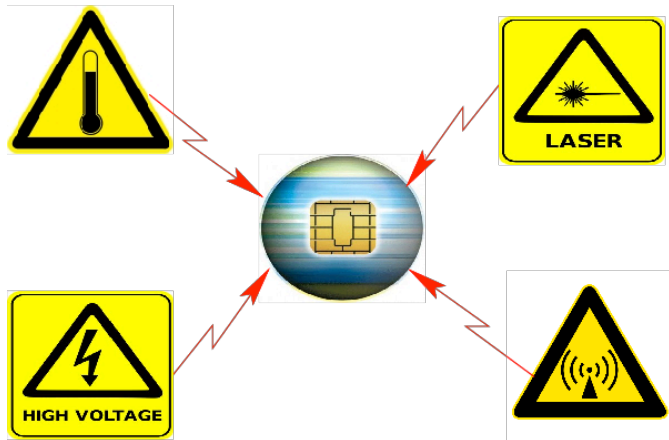
In Collaboration With

- *Dirmanto Jap and Jakub Breier* – **NTU, Singapore**
- *Sayandeep Saha, Sikhar Patranabis and Debdeep Mukhopadhyay* – **IIT Kharagpur**
- *Zhang, Fan, Xiaoxuan Lou, Xinjie Zhao, Ruyi Ding, Samiya Qureshi, and Kui Ren* -- **Zhejiang University, China**
- *Wei He* -- Shield Laboratory, **Huawei International Pte. Ltd., Singapore**

Table of Contents

1. Introduction to Fault Attacks
2. Persistent Fault Analysis (PFA)
3. PFA on Fault Countermeasures
4. Combined SCA + DFA
5. Conclusions

Fault Injection Attacks (FIA)



What is FIA?

- Physical Attacks
- Actively disturbs functioning of the target
- Exploits erroneous behavior

Injection Methods

- Global/Low-Cost/Low-Precision
 - Clock/Voltage glitch, temperature
- Local/High-Cost/High-Precision
 - Laser, Electromagnetic, Ion Beam

Impacts

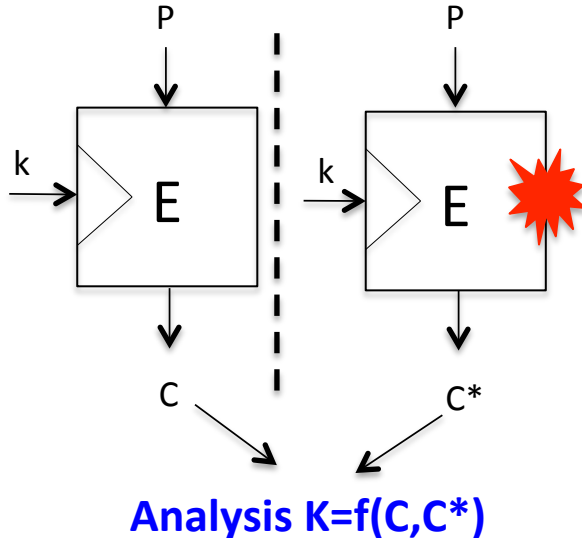
- Duration
 - Transient or Harmonic
- Effects
 - Data or Flow Modification
- Objectives
 - Corrupt computation, bypass security checks

Fault Models

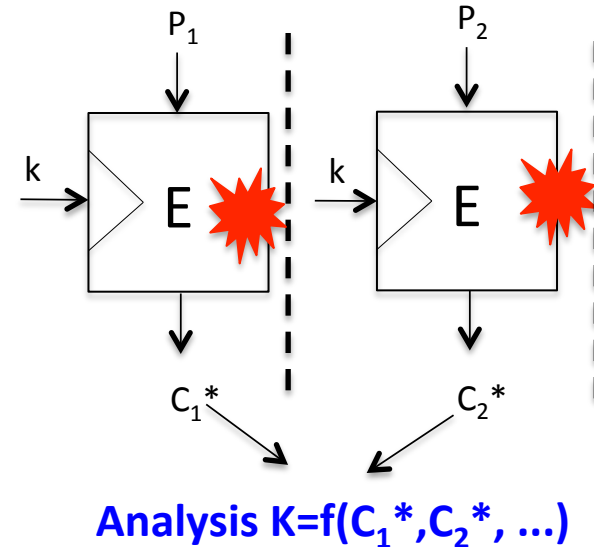
- **Single/multiple bit-flip** – a target variable was altered either by single or multiple bit flip.
- **Random byte fault** – Some bits of a byte are flipped. No-precise multi-bit flip.
- **Instruction skip** – One or several instructions were not executed (for software)
- **Stuck-at fault** – target variable stuck at-0/1

Fault Analysis

- Differential Fault Analysis (DFA)
- Usually few ciphertext pair
- Control over plaintext needed



- Statistical Fault Analysis (SFA)
- Need several ciphertext
- Several variants exist



Fault Countermeasures

- Two principle approaches
- Detection
 - Incremental
 - Sensors to detect physical condition
 - Redundancy to detect data modification
- Prevention
 - Provable
 - Infect/Correct fault

Limitations of SoA

- Very **tight time synchronization** on the round calculation and the injection timing
- Very **complicated analysis** due to the random value and the fault propagation
- **May not work** if there are **countermeasures** against fault attacks

Revisiting Fault types

- **Transient:** Affect one encryption
- **Permanent:** Always present
- **Persistent¹:** Hybrid model between **transient** and **permanent**. Persist over several encryptions but disappears on reboot. Typically targets stored constants (ex. Sbox in memory)

¹Persistent Fault Analysis. CHES 2018

Adversary Model

- Block cipher with **serial** implementation
- Common Sbox as **look-up table**
- **Persistent fault injected** in one Sbox element
- Victim encrypts **n plaintext** with faulty Sbox
- Adversary can observe the **n ciphertext**
- No control on plaintext, except **varying plaintext**

Persistent Fault Analysis: Main Idea

Stage 1: Fault Injection

12	5	6	11
9	0	10	13
3	14	15	8
4	7	1	2

S-box

Fault Injection

12	5	6	11
9	0	8	13
3	14	15	8
4	7	1	2

S-box *

* persists until refreshed

Stage 2: Encryption

n plaintext

S-box *

key k



n ciphertext

collected



adversary

Stage 3: Fault Analysis

Ciphertext	Probability
$10 \oplus k$	0
$8 \oplus k$	2/16
else	1/16

S-box* output	Probability
10	0
8	2/16
else	1/16

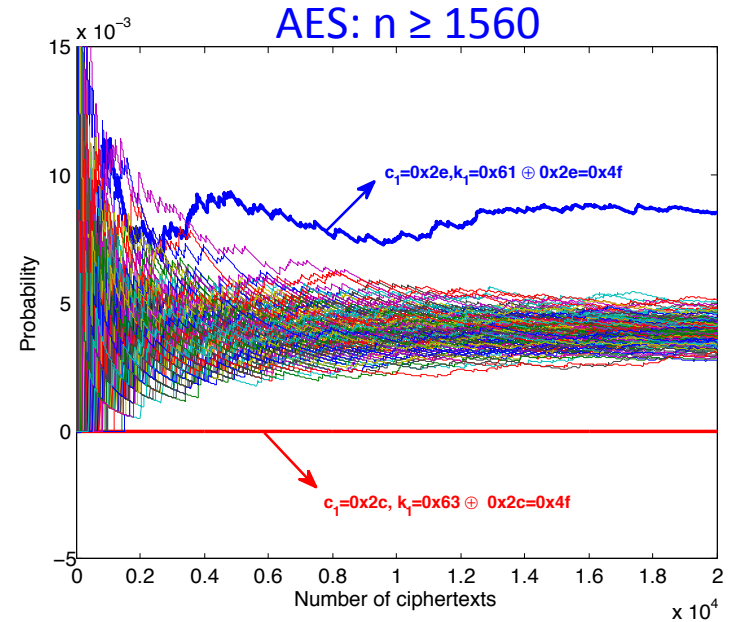
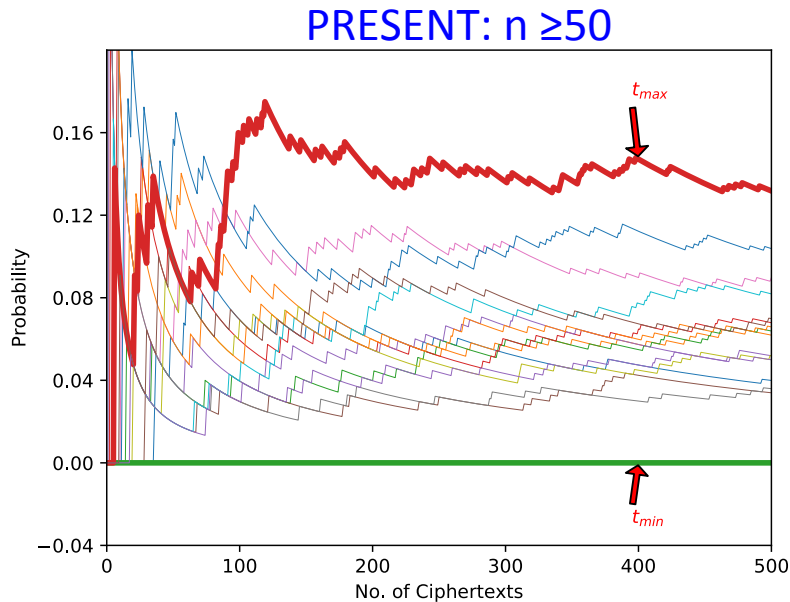
Input	Probability
all	1/16

key k exposed!

PFA: Modus Operandi

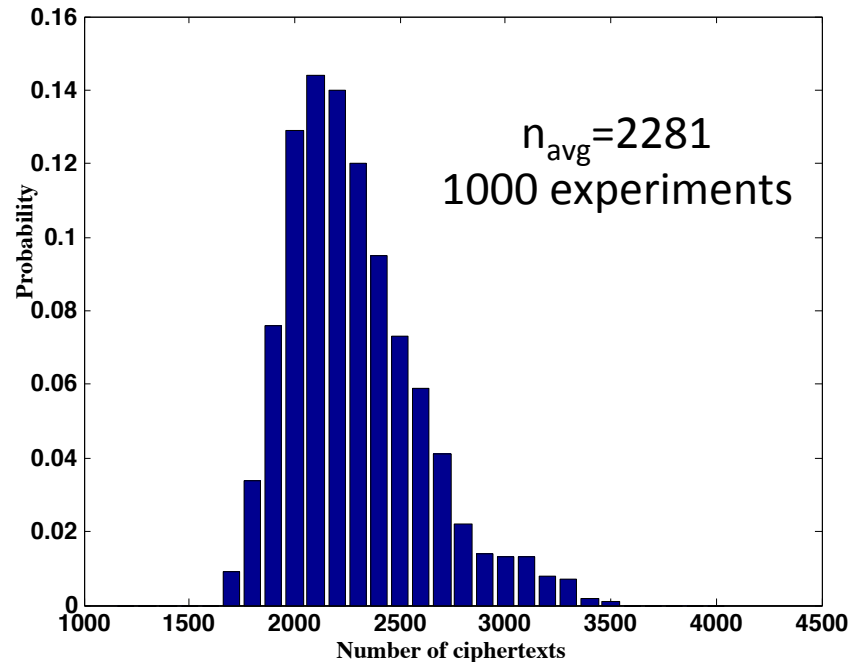
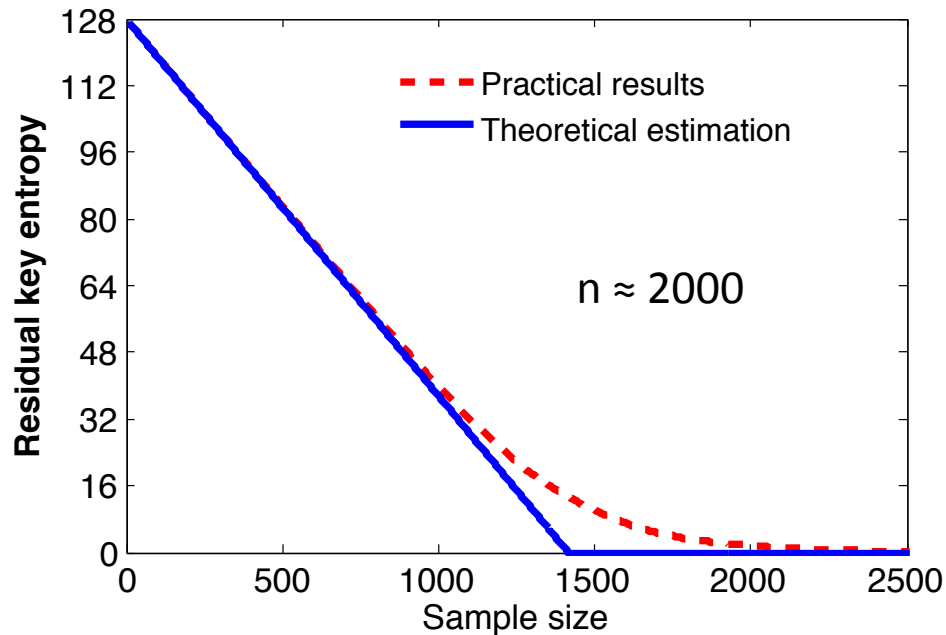
- Statistical analysis on last round with ciphertext only
- Fault changes one element $x \rightarrow x^*$ in Sbox (lets say 4X4 Sbox)
- Expectation $E(x) = 0$, $E(x^*) = 2/16$, $E(y \neq (x, x^*)) = 1/16$
- Three analysis strategies:
 - $t_{\min}$: find the missing value in Sbox table (x). Then $k = t_{\min} \oplus x$;
 - $t \neq t_{\min}$: find values t where $t \neq t_{\min}$ and eliminate candidates for k;
 - $t_{\max}$: find the value with max probability (x'). Then $k = t_{\max} \oplus x^*$
- No. of ciphertext n can be determined by coupon collector's problem
- x, x^* can be brute-forced if not known

PFA on PRESENT and AES



n = Minimum no of ciphertext needed by
coupon collector's problem

Practical PFA on AES



Comparison vs Other Fault Attacks



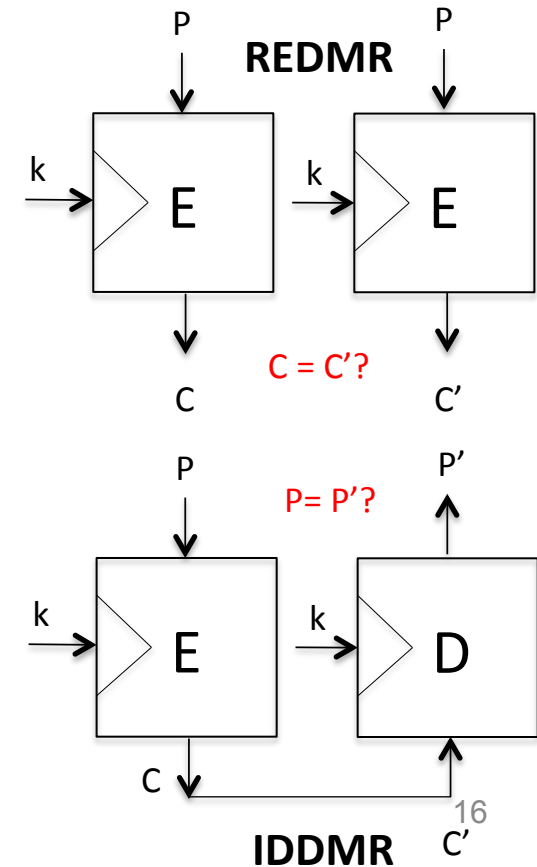
- (1) The attack is **not differential** in nature and thus the control over the plaintext is not required.
- (2) The adversary **does not necessarily need live synchronization**
- (3) The fault model remains **relaxed** (no biased faults needed)
- (4) PFA can also be applied in **multiple fault setting**
- (5) PFA can **bypass some redundancy based countermeasures**
- (6) An adversary can always inject the persistent fault **before the victim is switched to the sensitive mode**



- (1) It needs **higher number of ciphertexts** as compared to DFA
- (2) Persistent faults can be **detected by some built-in health test** mechanism or **fault counters**.

Dual Modular Redundancy (DMR) Countermeasure

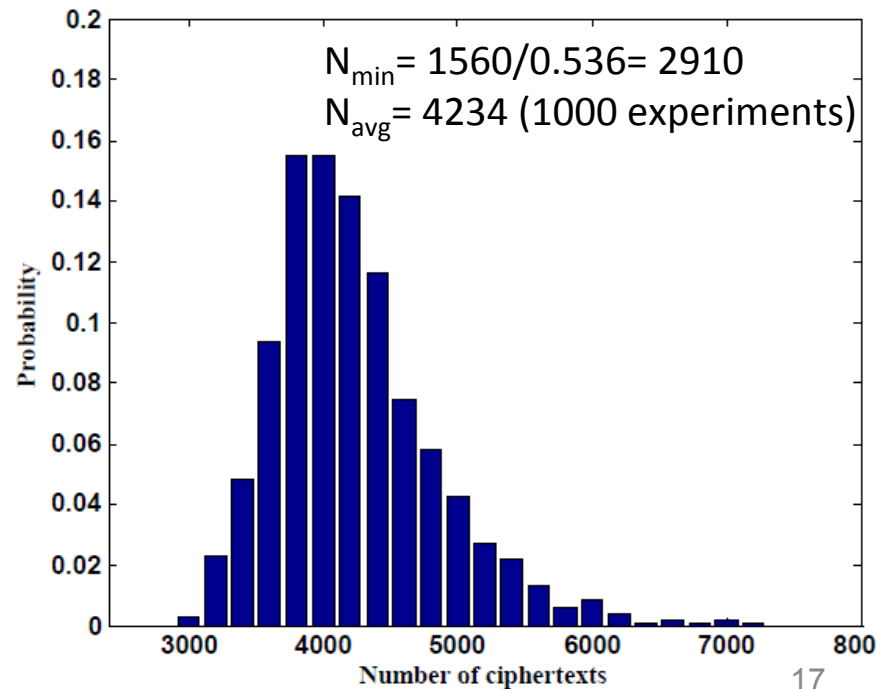
- Compute twice and compare (REDMR)
- Compute forward-inverse and compare (IDDMR)
- If $\neq$
 - **NCO**: No Ciphertext output
 - **ZVO**: Zero Value output
 - **RCO**: Random Ciphertext output
- Provably secure against single fault
- Adversary can either target the encryption or comparison but not both
- REDMR **broken by design** if same S-box is used
- Lets target IDDMR, more difficult of the two



Attacking IDDMR with NCO/ZVO

- Faulty outputs are suppressed
- Some output will be **not affected** by fault
- Probability p of correct output is $f(x,k)$
- p for AES

$$p = \left(1 - \frac{1}{256}\right)^{160} = 0.5346$$
- Adversary roughly needs **n/p ciphertext**



Attacking IDDMR with RCO

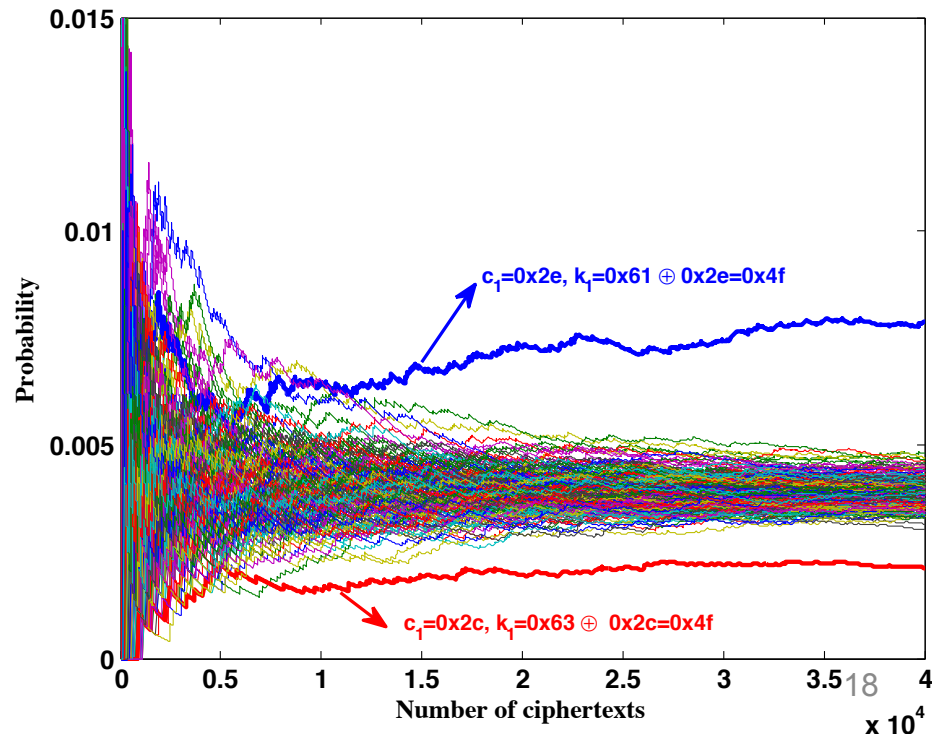
- Faulty output is **replaced by uniformly random**
- Slight difference in distribution of random output and correct ciphertext
- The **bias can be detected** with more ciphertext (n)

$$Pr(y = x) = 0 \times p + \frac{1}{256} \times (1 - p) = \frac{0.4654}{256}$$

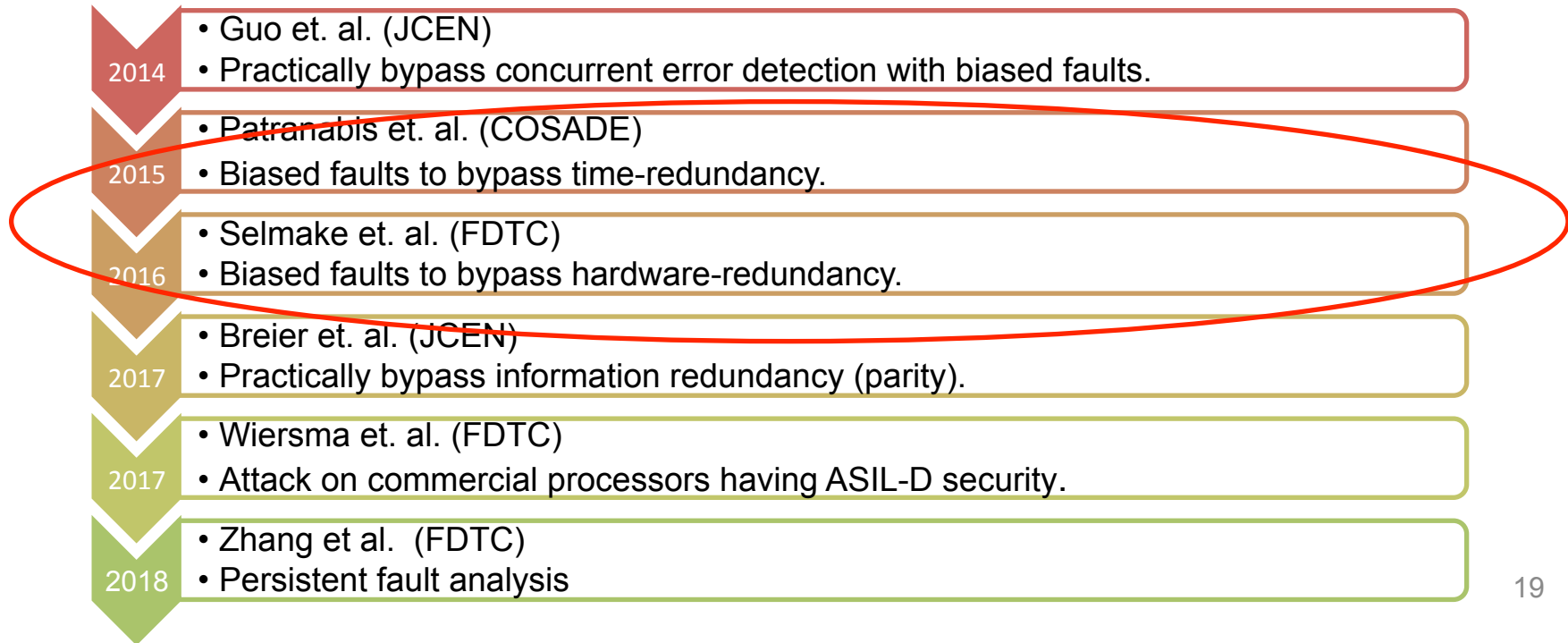
$$Pr(y = x^*) = \frac{2}{256} \times p + \frac{1}{256} \times (1 - p) = \frac{1.5346}{256}$$

$$Pr(y \neq x \wedge y \neq x^*) = \frac{1}{256} \times p + \frac{1}{256} \times (1 - p) = \frac{1}{256}$$

- Roughly **$n \approx 10000$** resulted in attack success



Attacks Against Redundancy



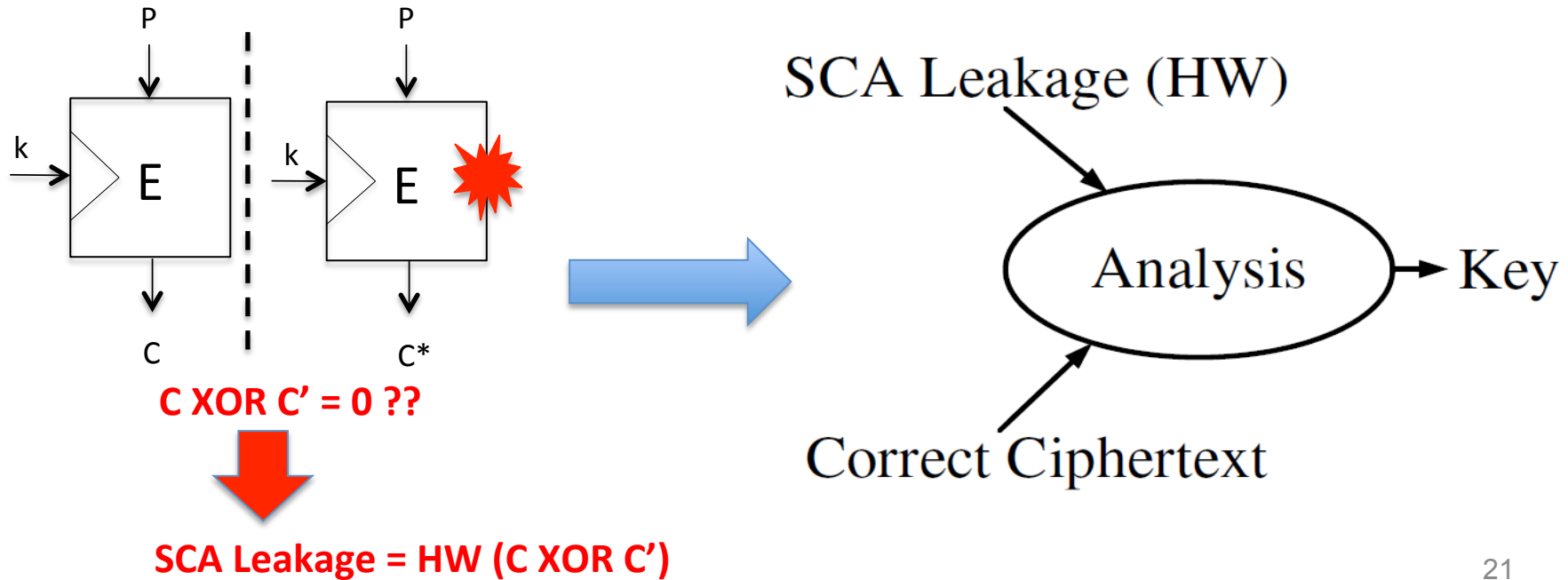
Advancing State of the Art

- Previous Works
 - Bypass the
- Our Proposal
 - Exploit the

!!!EVEN COUNTERMEASURE LEAK!!!

- Corrupt all computation branches
i.e. > 1 fault injection
- (relaxed model)
- Only 1 fault injection
- Side-channel assisted

Combined SCA+DFA on REDMR



The Attack Idea

- What we have and what we don't :

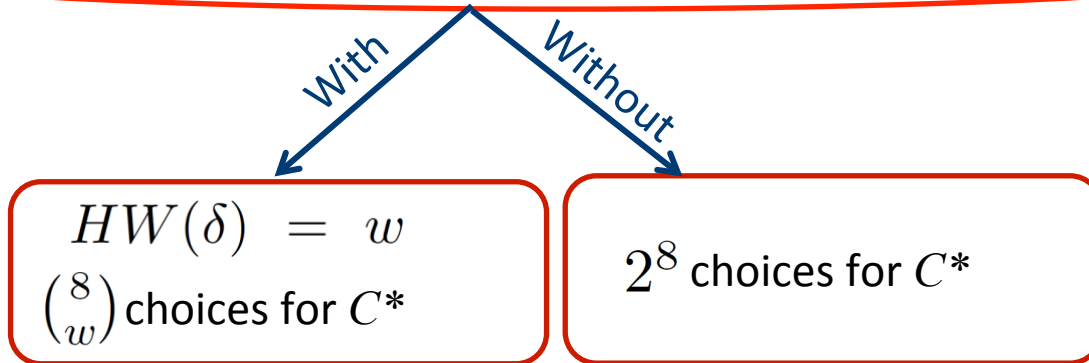
• Correct ciphertexts: C  Known

• Faulty Ciphertexts: C^*  Unknown

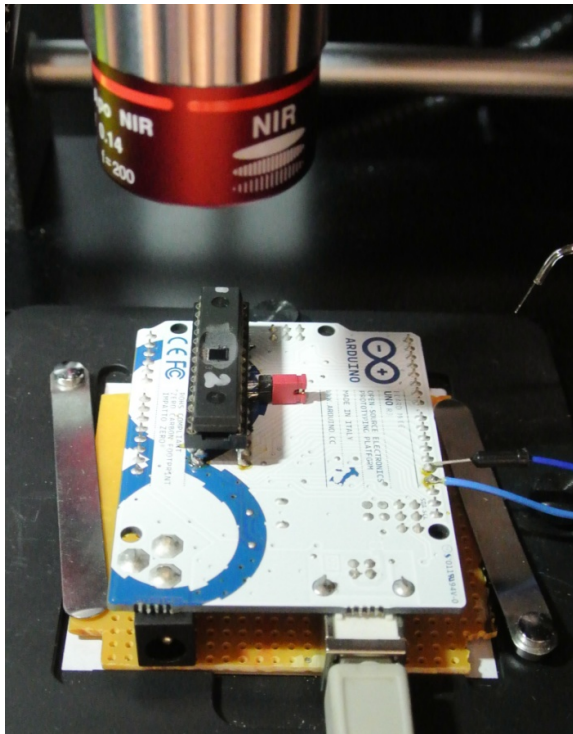
• $HW(\delta) = HW(C \oplus C^*)$  Known n (bitwise)

$W=8$,
 C^* Known

Less choices
for extreme w

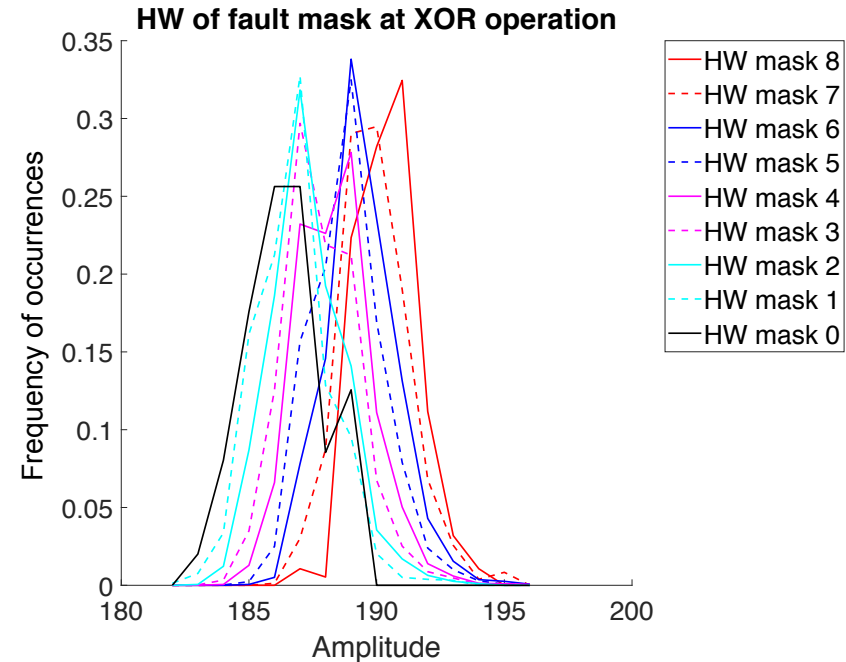
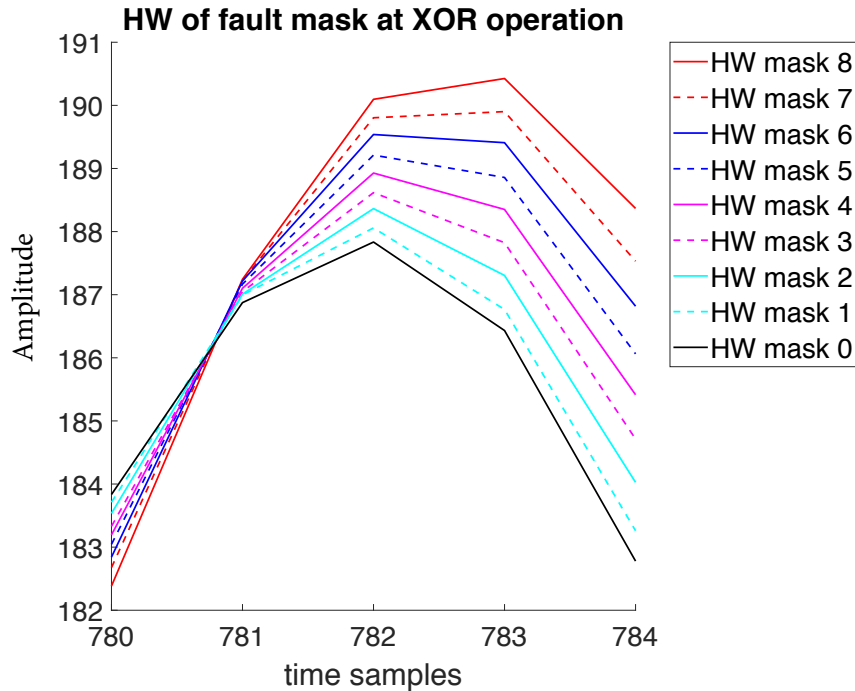


Attack Setup

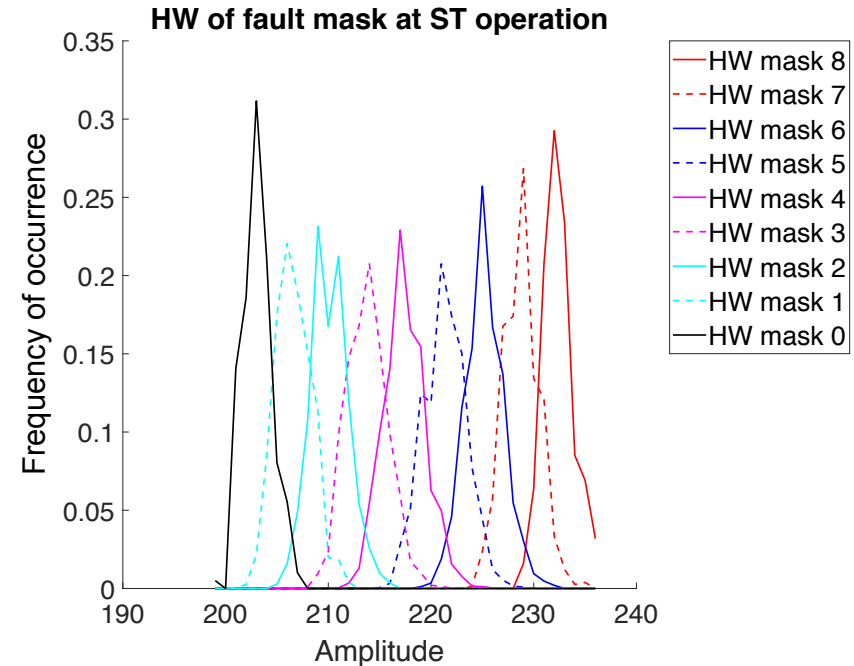
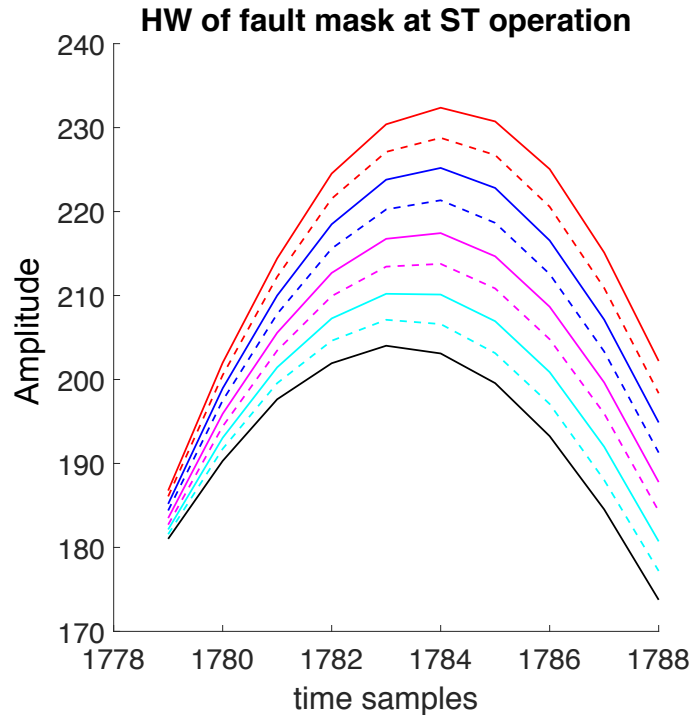


- Near-infrared diode pulse laser
- Maximum output power of 20 W
- For the experiments, 20x magnifying objective lens was used
- As a DUT, ATmega328P was used – an 8-bit microcontroller running at 16 MHz
- Chip was depackaged from the backside to be accessible by the laser
- Total area vulnerable to experiments was <1% of the entire chip area
- Reproducibility of faults was near to 100% with the same laser settings

Observing $C \oplus C'$ (Sniff XOR)



Observing $C \oplus C'$ (Sniff ST)



Practical validation

Cipher	Code Size (bytes)	T_{ENC}	N_{EXP}	$(\mathcal{E} , \mathcal{F} , \mathcal{R})$
AES-128	7570	0.326	$2^{26.98}$	$(2^{43}, 2^{25}, 1)$
PRESENT-80	7110	4.01	$2^{23.36}$	$(2^4, 4, 1)$

$(|\mathcal{E}|, |\mathcal{F}|, |\mathcal{R}|) = (\text{Computation complexity, No. Of Faults, Remaining Key Space})$

2^{25} Practical injections/day possible with our setup

Conclusions

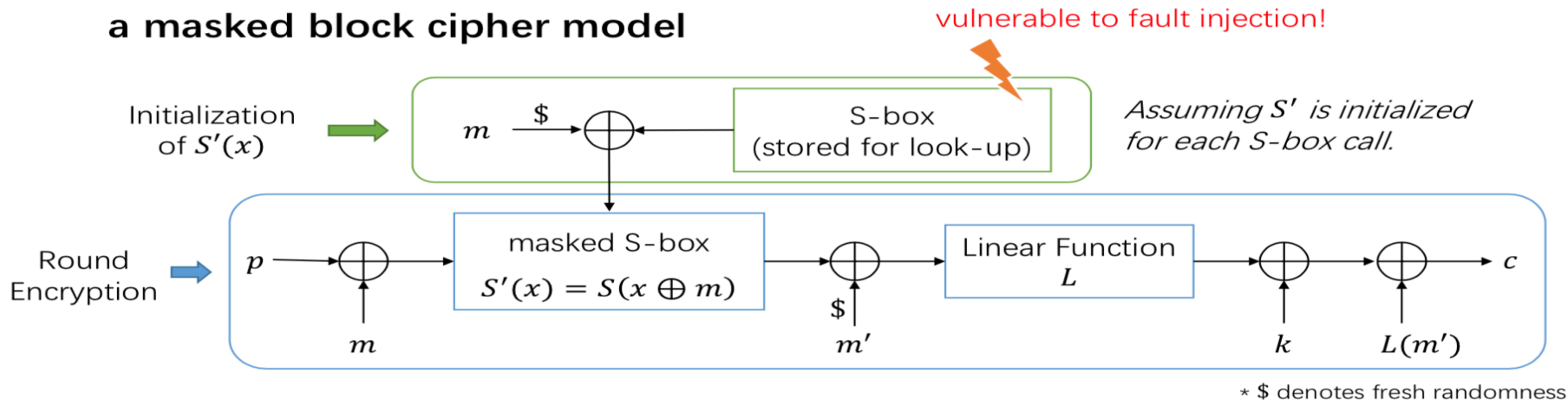
- Persistent Fault Analysis (PFA)
 - A novel attack on general block ciphers
 - Defeat popular fault countermeasures
 - Can work with multiple faults
 - One one fault injection required
- Combined SCA+DFA
 - Pushing the limits
 - Exploiting leakage from the countermeasures

Future Works

- Attack higher-order masking with PFA
 - Accepted at DATE 2019
 - Target public implementation of higher-order masking with one fault
- Analyze combined countermeasure
- Develop countermeasures against PFA
- Application on PKC/PQC?

PFA on Masking

a masked block cipher model



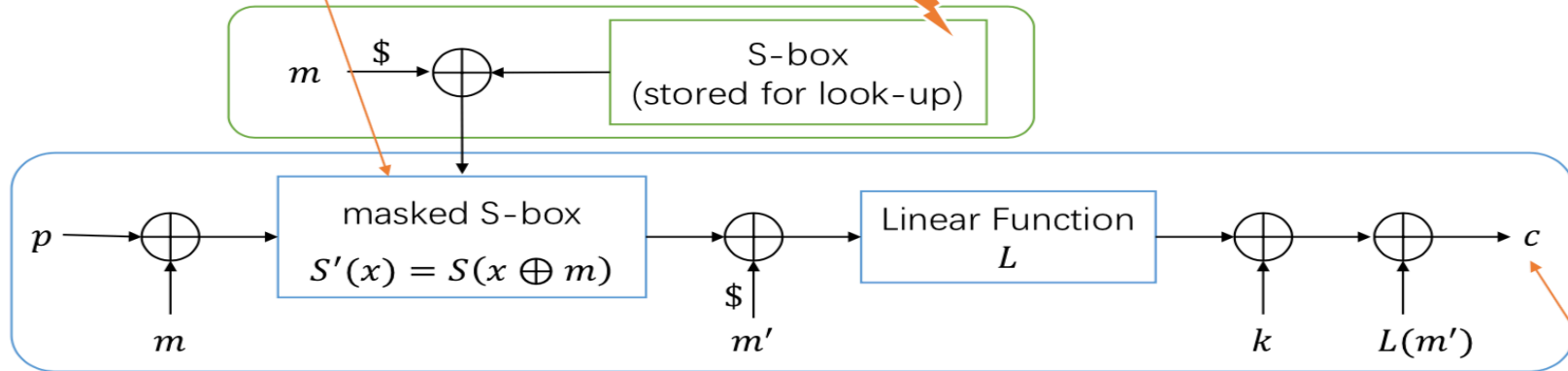
$$c = (L(S'(p \oplus m) \oplus m') \oplus k) \oplus L(m')$$

And for higher order d masking, we have $m = m_1 \oplus m_2 \oplus \dots \oplus m_d$; the same with m' .

PFA on Masking

② Faulty Initialization of S' :
 $x \oplus m$ (missing) $\rightarrow x' \oplus m$ (doubled)

① Fault Injection:
 original value $x \rightarrow$ faulty value x'



Key Exposure !

equivalent to attacking an
unmasked implementation

③ Deduction:

$c_* = L(x \oplus m') \oplus L(m') \oplus k = L(x) \oplus k$ is missing.
 $c'_* = L(x' \oplus m') \oplus L(m') \oplus k = L(x') \oplus k$ is doubled.

Thank You !!!

Questions ???